

Faro — Manual técnico

Cómo funciona por dentro: arquitectura, algoritmos, fórmulas y decisiones de diseño.

Escrito para quien va a mantener este código. Cada afirmación apunta al archivo y la línea donde vive, y lo que es un supuesto se dice que lo es.

1. Qué es esto y cómo está partido

Faro convierte el historial de ventas de un distribuidor en la decisión de compra del día: qué pedir, cuánto, a quién y cuándo hay que ponerlo para que llegue antes de quedarse sin producto. Este documento describe cómo lo hace por dentro. Está escrito contra el código, con archivo y línea, y cuando un número es un supuesto lo dice.

La cadena completa

Todo el producto es una cadena de cinco eslabones. Cada uno recibe algo medido y entrega algo más útil, y en cada empalme hay un supuesto que conviene conocer.

CÓMO FUNCIONA

- Historial de ventas (CSV/Excel/SQL) → una tabla canónica con sku, fecha y demanda.
- Por SKU: clasificación de la serie, selección de modelos, entrenamiento con validación hacia adelante, elección de campeón.
- Pronóstico con intervalo → demanda diaria esperada durante el plazo de entrega.
- Contra el stock de hoy: días de cobertura → señal del semáforo → cantidad recomendada.
- Orden de compra, envío al proveedor, recepción y aprendizaje del plazo real.

LO QUE NO SIGNIFICA

— El último eslabón es el que cierra el círculo: sin registrar la orden y su recepción, el plazo real del proveedor nunca se aprende y el sistema sigue planificando sobre lo configurado.

Las tres capas

[ForecastingCore/](#) · [backend/](#) · [Frontend/](#)

Separación estricta y con una razón: el motor puede venderse, probarse y reemplazarse sin tocar el producto.

ForecastingCore/

TODA la inteligencia de pronóstico. No conoce la base de datos, no conoce tenants. Recibe un diccionario de configuración y devuelve diccionarios y DataFrames.

backend/

API FastAPI multi-tenant. Orquestación pura: no hace ML, y no puede importar pandas ni numpy fuera de tres módulos — lo verifica un test.

Frontend/

Next.js 14. Sin lógica de negocio: pide, muestra y traduce.

LO QUE NO SIGNIFICA

— El motor NUNCA escribe en base de datos. Quien persiste es backend/workers/runner.py, que es también el único puente entre las dos capas.

Qué decide qué

Cuatro decisiones mandan sobre todo lo demás. Si algo del producto sorprende, casi siempre es una de estas.

La clasificación de la serie

Decide qué modelos se entrenan. Una serie corta (< 20 observaciones) se descarta del todo.

El campeón por SKU	Decide qué pronóstico se usa río abajo. La regla vive en una constante compartida entre el motor y el backend, literalmente, porque cuando divergieron las dos capas discreparon en 8 de 13 productos.
El plazo de entrega	Decide los tres umbrales del semáforo Y la demanda del período de reposición. Es el número más sensible del sistema y su valor por defecto es un supuesto de 15 días.
El nivel de servicio	Decide el colchón. Por defecto 0.95, resoluble por SKU, proveedor, categoría o global.

Cómo leer el resto del documento

Cada tema trae lo mismo: qué es, cómo funciona, la fórmula donde la hay, y — la sección que más importa — qué NO significa.

LO QUE NO SIGNIFICA

- Las advertencias no son autocrítica decorativa. Son las cosas que un ingeniero encuentra en la primera semana; encontrarlas sin aviso es lo que hace que el resto del documento parezca publicidad.
- Donde una constante está fija en el código y no es configurable, se dice. Donde un parámetro existe pero nadie lo lee, también.

2. Ingesta: del archivo del cliente a una serie modelable

Faro no pide un formato. Lee el export que el cliente ya tiene — CSV o Excel, con separador y codificación variables, encabezados en español y fechas dd/mm/yyyy — y lo convierte en una tabla canónica. Esta es la parte que más trabajo de campo tiene y la que más barato es subestimar: casi todo archivo real llega roto de alguna forma.

Lectura del archivo

[forecasting_core/data/loader.py:109](#)

Un solo punto de entrada para CSV, XLSX, XLS, Parquet, JSON y SQL. Lo que parece trivial es donde se pierden los archivos reales.

CÓMO FUNCIONA

- El separador se olfatea del encabezado entre coma, punto y coma, tabulador y barra vertical (loader.py:17).
- Las codificaciones se prueban en orden: utf-8-sig, cp1252, latin-1 (loader.py:37). El BOM de Excel y el cp1252 de Windows son la norma, no la excepción.
- Una columna de fechas inequívocamente dd/mm/yyyy se lee con día primero (loader.py:62). Si es ambigua, no adivina.

LO QUE NO SIGNIFICA

— Un archivo con NUL se rechaza en backend/dataframes/io.py, no acá: pandas trunca la celda en el NUL sin avisar, y un truncado silencioso es peor que un rechazo.

Esquema canónico

[forecasting_core/data/canonical.py:49](#)

Catorce campos canónicos a los que se mapean las columnas del usuario. Tres son obligatorios: sku, date, demand. El resto tiene un valor por defecto que se difunde a toda la columna cuando el archivo no lo trae.

sku, date, demand	Obligatorios. Sin ellos no hay serie.
store	Por defecto «Tienda única». La llave de serie es sku store, con U+2502 de separador (canonical.py:66).
region	Por defecto «Sin región».
inventory	Por defecto 0. Si el archivo la trae de verdad, activa la recuperación de demanda censurada.
lead_time	Por defecto 15 días (DEFAULT_LEAD_TIME_DAYS, canonical.py:23). Es un SUPUESTO, y el producto lo dice en pantalla.
price, cost, regular_price, promo_price	Por defecto None.
promo, promo_type, discount	Por defecto False, «Sin promoción», 0.0.

LO QUE NO SIGNIFICA

- Los 15 días de lead time por defecto son el supuesto más caro del sistema: un importador con 45 días de tránsito reordena un mes tarde y nada falla visiblemente.

Relleno de huecos y valores atípicos

[forecasting_core/validation/auto_correct.py:100](#)

Dos correcciones automáticas antes de modelar, y una distinción que importa: rellenar un TARGET nulo no es lo mismo que rellenar una FECHA faltante.

CÓMO FUNCIONA

- Target nulo: por grupo, ffill → bfill → fillna(0) (auto_correct.py:126). Siempre activo.
- Atípicos: recorte por SKU a $[Q1 - 3 \cdot IQR, Q3 + 3 \cdot IQR]$ (auto_correct.py:139). El pipeline lo pasa como clip_outliers=True incondicionalmente (pipeline.py:279).
- Fechas faltantes: NO las rellena el motor. Las estrategias (cero, ffill, interpolar, dejar) viven en el catálogo de la compuerta (data/gate.py:107) y las aplica backend/workers/runner.py; el valor por defecto es «dejar».

LO QUE NO SIGNIFICA

- auto_correct.py:104 declara un parámetro fill_missing_dates y NUNCA lo usa. Es código muerto: quien lo lea y suponga que rellena fechas se equivoca.
- DataQualityChecker.clip_outliers (quality.py:223) existe y el pipeline no lo llama. El mismo factor 3.0 sí se usa para CONTAR atípicos en el puntaje de calidad.

Demanda censurada

[forecasting_core/data/censoring.py](#)

Un día sin ventas porque no había producto en bodega no es un día de demanda cero. Si el cliente mapeó una columna real de inventario, esas observaciones se levantan; si no la mapeó, este paso es un no-op.

LEVEL_WINDOW	28 — ventana para estimar el nivel de demanda no censurada.
MIN_UNCENSORED	14 — mínimo de observaciones sin censura para estimar.
MAX_LIFT_FACTOR	3.0 — techo del ajuste. Nunca multiplica una venta por más de tres.
demand_censored	Columna bandera que queda en la tabla.

LO QUE NO SIGNIFICA

- Las observaciones solo se LEVANTAN, nunca se bajan. Es una corrección de sesgo hacia arriba y hay que saberlo al leer un pronóstico.

Clasificación de series

[forecasting_core/data/quality.py:80](#)

Cada SKU recibe etiquetas — es MULTI-ETIQUETA, una serie puede ser estacional y volátil a la vez. De esto depende qué modelos se le entrenan.

CÓMO FUNCIONA

- Si $n < \text{min_history} (20) \rightarrow \{\text{short}\}$ y se corta ahí, sin evaluar nada más (quality.py:101).
- Si la proporción de ceros $\geq 0.4 \rightarrow \text{intermittent}$.
- Si $\text{CV} = \text{std}/(|\text{media}|+1\text{e-}8) \geq 1.5 \rightarrow \text{volatile}$.
- Si hay largo suficiente, STL con $\text{period}=7$: $\text{fuerza} = \max(0, 1 - \text{var}(\text{resid})/(\text{var}(\text{seasonal}+\text{resid})+1\text{e-}8))$; $> 0.3 \rightarrow \text{seasonal}$. Cualquier excepción se traga y no marca estacional.
- Sin ninguna bandera $\rightarrow \text{stable}$.

Fuerza estacional

$\text{strength} = \max(0, 1 - \text{var}(\text{resid}) / (\text{var}(\text{seasonal} + \text{resid}) + 1\text{e-}8))$

STL robusto, período 7 por defecto (training.seasonal_period).

Puntaje de calidad

$100 - 30 \cdot (n < 20) - \min(\text{missing} \cdot 2, 20) - \min(\text{outliers} \cdot 3, 15) - \min(\text{zero_ratio} \cdot 20, 20)$

Piso en 0, publicado dividido entre 100 (quality.py:266).

LO QUE NO SIGNIFICA

- `filter_valid_skus` descarta todo SKU con menos de 20 observaciones. Un catálogo nuevo puede perder la mitad de sus productos y eso es correcto, pero hay que explicárselo al cliente.
- Existe una función legado `classify_series` (quality.py:53) con umbrales propios que el pipeline NO usa.

Granularidad y agregación

[forecasting_core/data/resampler.py:19](#)

La estrategia por defecto es «native»: no toca nada. Con «aggregate» se remuestrea a una frecuencia objetivo sumando el target y agregando las demás columnas según un diccionario (el inventario se lleva con min).

LO QUE NO SIGNIFICA

- Limitación declarada en el propio código (pipeline.py:181): al remuestrear solo se usa la llave de grupo PRIMARIA, así que una configuración con dos llaves suma a través de la secundaria.
- Hay dos escaleras distintas de detección de frecuencia con cotas distintas — `profiler.py:911` y `profiler.py:935` — más `infer_freq_days` en `quality.py:27`. No están unificadas.

3. Pronóstico: de la serie a un número con intervalo

El motor vive completo en ``ForecastingCore/`` y no toca la base de datos: recibe una configuración, devuelve diccionarios y DataFrames, y es el backend quien persiste. Entrenar es por SKU, con validación hacia adelante, y el campeón se elige con una regla que el backend comparte literalmente — cuando divergieron, las dos capas discrepaban en 8 de 13 productos.

Punto de entrada y orden del pipeline

`forecasting_core/engine.py:68` → `forecasting_core/pipelines/pipeline.py:201` · `run`

``ForecastEngine`` expone una API fluida; ``Pipeline.run()`` hace el trabajo en 16 etapas. El único puente desde el backend es ``backend/workers/runner.py:1312``.

CÓMO FUNCIONA

- Lectura del archivo → validaciones (en modo AVISO, nunca abortan) → autocorrección → demanda censurada.
- Calidad por SKU, clasificación de series y descarte de series cortas.
- Enrutamiento de modelos → ingeniería de características → baselines.
- Entrenamiento por SKU con validación hacia adelante, modelos cuantílicos, modelo global, modelos estadísticos.
- Ensamble ponderado → tabla de métricas → pronósticos futuros → recomendaciones de inventario → registro de la corrida.

LO QUE NO SIGNIFICA

- La reconciliación jerárquica (`bottom_up` / `top_down` / `mint`) existe pero está APAGADA por defecto: `hierarchy.levels` llega vacío (`config.py:164`).
- El motor nunca escribe en base de datos. `ModelRegistry.log_run` escribe un JSON en disco (`registry.json` por defecto).

Enrutamiento de modelos

`forecasting_core/training/router.py:30`

La clasificación de la serie decide qué modelos tiene sentido entrenarle. La regla que gobierna todo: el enrutamiento solo puede ESTRECHAR la selección del usuario, nunca ampliarla.

La regla

`run = assigned & declared # intersección, nunca unión`

``declared`` es la selección del usuario, verbatim. Si la intersección queda vacía, se cae a ``declared`` completo en vez de dejar el SKU sin modelo (`router.py:80`).

short	naive, seasonal_naive
intermittent	croston, ets
seasonal	prophet, ets, lightgbm, sarimax, lstm

stable	lightgbm, xgboost, arima, sarimax, lstm
volatile	lightgbm, xgboost, prophet, arima, ets, sarimax

LO QUE NO SIGNIFICA

- Las etiquetas se unen antes de intersecar: una serie {seasonal, volatile} recibe la unión de ambas filas.
- HUECO REAL: `naive` y `seasonal\_naive` aparecen en la tabla para series cortas, pero NO existen como modelos entrenables en ModelFactory — solo como baselines. Una serie corta siempre cae en el fallback de intersección vacía.
- `global\_lgbm` está en UNIVERSAL_MODELS y se salta el estrechamiento, pero sigue condicionado a que el usuario lo haya declarado.

Modelos implementados

[forecasting_core/models/factory.py:27](#) · ModelFactory

Nueve modelos entrenables más tres baselines. El paralelismo está en el bucle de SKUs, por eso cada modelo corre con `n_jobs=1`.

lightgbm / xgboost	<code>n_estimators</code> 300, <code>learning_rate</code> 0.05 por defecto. Los únicos con espacio de búsqueda en Optuna.
global_lgbm	Un LightGBM sobre todo el catálogo, multi-horizonte directo (el horizonte es una característica). 400 árboles, 63 hojas. Matriz apilada con techo de 2.000.000 de filas.
arima	<code>statsmodels</code> , orden (5,1,2). Reajusta sobre la serie completa para el futuro; <code>conf_int(alpha=0.2)</code> da p10/p90.
sarimax	Orden (1,1,1), estacional (1,1,1,período). Solo corre si el usuario lo seleccionó explícitamente.
prophet	Estacionalidad anual y semanal activas, diaria apagada.
ets	Tendencia aditiva; estacionalidad aditiva solo si hay 2 períodos completos y toda la serie es positiva.
croston	Croston clásico, <code>alpha</code> 0.1. No es SBA ni TSB.
lstm	TensorFlow OPCIONAL: si no está instalado devuelve vacío con un aviso. Ventana 14, 50 épocas, parada temprana.
baselines	<code>naive</code> , <code>seasonal_naive</code> , <code>historical_avg</code> . Se puntúan para comparar y están EXCLUIDOS de ser campeón.

LO QUE NO SIGNIFICA

- El esquema de sesión que expone la configuración (`config.py:348`) lista 8 modelos y OMITE `global_lgbm`, mientras que ModelFactory sí lo tiene. La selección por defecto del backend sí lo incluye.
- Optuna está apagado por defecto y solo tiene espacio de búsqueda para `lightgbm` y `xgboost`; cualquier otro nombre devuelve parámetros intactos.

Características y la guardia de fuga

[forecasting_core/features/engineer.py:47](#)

Calendario, Fourier opcional, rezagos, diferencias, ventanas móviles y EWM. Todo lo que mira hacia atrás se construye sobre `shift(1)` para que la fila no pueda reconstruir su propio objetivo.

CÓMO FUNCIONA

- Calendario: 19 columnas, el mismo constructor para entrenamiento e inferencia. Feriados por país desde la librería ``holidays``, país por defecto CO; si falta la librería degrada a Pascua y Navidad, nunca a un conjunto vacío.
- Rezagos [1, 7, 14]; diferencias [1, 7] calculadas sobre `shift(1)`; ventanas [7, 14, 28] con media, desviación, mínimo, máximo y CV.
- El dropna de calentamiento corre SOLO sobre columnas generadas y el objetivo — un dropna general llegó a vaciar datasets enteros.

Guardia de fuga (trainer.py:266)

```
leak_cols = [c for c in X if is_numeric(X[c]) and X[c].astype(float).equals(y)]
```

Cualquier columna idénticamente igual al objetivo se descarta con un aviso que nombra el SKU. Apunta al alias canónico ``demand``, que es copia literal de la columna del usuario.

LO QUE NO SIGNIFICA

- ``_drop_unservable_features`` quita columnas que existen en el histórico pero no se pueden reproducir para una fecha futura (inventory, price, cost).
- PARCIALMENTE INERTE: `validation/leakage.py` se llama sin ``feature_columns``, así que solo dispara la comprobación de fechas desordenadas. Las comprobaciones de fuga por correlación (> 0.999) son inalcanzables desde el pipeline.

Validación y elección del campeón

[forecasting_core/pipelines/pipeline.py:794](#) · [_select_champions](#)

Validación hacia adelante con ventana expansiva y un HUECO igual al horizonte, para que la ventana de entrenamiento nunca toque datos que el pronóstico de producción no habría tenido.

CÓMO FUNCIONA

- ML por SKU: 3 cortes por defecto, entrenamiento mínimo del 40% de la serie, hueco = horizonte. Si no hay cortes viables, cae a una partición cronológica simple.
- Dos puntajes por modelo a propósito: las métricas de corte son a 1 paso, y ``horizon_metrics`` corre el camino de inferencia REAL hacia adelante desde el último corte.
- Modelo global: origen rodante sobre el eje de fechas, reajustando en cada corte.
- Modelos estadísticos: una sola partición cronológica; su costo ya es un número a h pasos.

Métrica principal

$$\text{WAPE} = \frac{\sum |y - \hat{y}|}{(\sum |y| + 1e-8)}$$

Además: MAE, RMSE, sesgo, MAPE, sMAPE, pinball.

Costo asimétrico

```
cost = mean(sobrante + 3.0 * faltante)
```

DEFAULT_STOCKOUT_MULTIPLIER = 3.0. Quedarse sin producto cuesta el triple que sobrar.

Regla del campeón

```
metric = primera de (cost_horizon, cost, wape, mae) presente; champion = argmin(metric)
```

Se excluyen las filas de baseline: un pronóstico ingenuo nunca puede ser campeón. Sin desempate explícito.

LO QUE NO SIGNIFICA

- Si el mejor baseline le gana a todos los modelos, el SKU se anota en ``outperformed_by_baseline`` y aun así recibe su mejor modelo real.
- El ensamble emite pronóstico pero NO tiene fila de métricas, así que nunca puede ser campeón.
- MASE y CRPS no están implementadas en ninguna parte.
- CHAMPION_METRIC_ORDER se comparte literalmente con `backend/inventory/service.py::best_model_by_sku`. Dejarlas divergir hizo que las dos capas discreparan en 8 de 13 SKUs.

Intervalos de predicción

[forecasting_core/inference/predictor.py:29](#)

Tres mecanismos distintos, y cuál te toca depende del modelo. Esto importa más de lo que parece: el inventario extrae su sigma de estos intervalos.

Aproximación normal	<code>bound = max(0, valor + z_q · std(residuos))</code> . Banco de residuos fuera de muestra cuando existe. Es el camino de LightGBM/XGBoost sin cuantílicos, y de ETS, Croston y SARIMAX.
Regresores cuantílicos	LightGBM/XGBoost entrenados en <code>q = 0.1/0.5/0.9</code> y predichos directamente.
Conformal por horizonte	Solo <code>global_lgbm</code> . Cuantiles empíricos de los residuos del backtest, SEPARADOS por horizonte, mancomunados en unidades escaladas. Mínimo 30 residuos por horizonte; por debajo, toma prestado del banco general.
Bandas propias	ARIMA (IC al 80%), Prophet (<code>yhat_lower/upper</code>), LSTM (Monte-Carlo Dropout, 30 pasadas).

LO QUE NO SIGNIFICA

- Si no hay datos de calibración, el conformal devuelve una banda DEGENERADA (`inferior = superior = valor`) en vez de inventar un ancho.
- ``enforce_monotonic`` repara cuantiles cruzados con un máximo corrido;
- ``enforce_horizon_monotonic`` hace no decrecientes las bandas acumuladas.
- ``lead_time_demand_quantile`` entrega el cuantil de la demanda ACUMULADA sobre el lead time: es lo que consume el punto de reorden.

4. Inventario: del pronóstico a la orden de compra

Acá vive el producto. El pronóstico es un insumo; lo que el cliente compra es «pide 78 unidades de esto hoy». Todo este capítulo son cuatro números encadenados — cobertura, señal, punto de reorden y cantidad — y la parte más importante es dónde cada uno deja de ser una medición y pasa a ser un supuesto.

Días de cobertura

[backend/inventory/service.py:1662](#) · `coverage_days`

Cuánto dura el stock que hay hoy al ritmo de venta pronosticado. Es el número del que cuelga todo lo demás.

Cobertura

```
coverage_days = current_stock / avg_daily si avg_daily > 0, si no 9999.0
```

`avg\_daily` es la media de los primeros `steps` puntos del pronóstico del MEJOR modelo del SKU, con $\text{steps} = \text{ceil}(\text{lead_time_días} / \text{días_por_período})$.

LO QUE NO SIGNIFICA

- NO ESTÁ EN DÍAS, pese al nombre. Está en períodos del grano de planificación activo: en una sesión semanal, «3» son 3 semanas. El campo autoritativo es `coverage\_unit`, que viaja al lado.
- El numerador es solo stock en mano. Las órdenes en tránsito NO entran acá (sí entran en la cantidad recomendada), así que un SKU con un camión en camino puede mostrar PEDIR_YA con cantidad 0.
- Demanda cero da el centinela 9999, que clasifica como SOBRESTOCK y se anula antes de salir: el cliente ve `coverage\_days: null, signal: SOBRESTOCK`.
- Si el mejor modelo no guardó puntos, cae silenciosamente a promediar TODOS los modelos entrenados.

El semáforo

[backend/inventory/service.py:1065](#) · `_calc_signal`

Toda la regla, literal, son cuatro líneas.

Señal

```
cobertura < lead_time*0.5 → PEDIR_YA cobertura < lead_time*1.2 → PEDIR_PRONTO cobertura < lead_time*3 → OK si no → SOBRESTOCK
```

Los multiplicadores 0.5 / 1.2 / 3 son literales en el código.

LO QUE NO SIGNIFICA

- NO hay configuración de umbrales por tenant ni por SKU. Lo único que varía por producto es el lead time.
- SIN_DATOS no sale de esta función: es la rama cuando falta el pronóstico o falta la fila de stock. En la vista por bodega también aparece cuando la participación de demanda de esa bodega es 0.
- La importación fuerza `lead_time_days >= 1`: un 0 colapsa los tres umbrales y pinta todo SOBRESTOCK.

— ``min_stock`` se guarda, se importa y se devuelve, pero NO lo lee ni la señal ni la recomendación. Es peso muerto en esta capa.

Cantidad recomendada

[backend/inventory/service.py:1119 · _calc_recommended](#)

Demanda del lead time más stock de seguridad, menos lo que ya tienes y lo que viene en camino. Con un piso de MOQ y una compuerta por señal.

Cantidad

```
raw = max(0, avg_daily.lead_time + safety_stock - current_stock - incoming) si moq > 0 y  
raw > 0: raw = max(ceil(raw), moq)
```

Después se fuerza a 0 salvo que la señal sea PEDIR_YA o PEDIR_PRONTO.

Punto de reorden

```
reorder_point = avg_daily.lead_time + safety_stock
```

Se publica, pero NO es el disparador: el disparador es la razón de cobertura de arriba.

Stock de seguridad — medido (preferido)

offsets acumulados del backtest de origen rodante, horizonte `ceil(lead_time)`, cuantil más cercano al nivel de servicio

Solo se usa si el modelo que produjo el riesgo es el mismo campeón del SKU; si no, se descarta.

Stock de seguridad — respaldo clásico

```
safety = z · σ · sqrt(lead_time)
```

σ se extrae del intervalo: $\sigma = (q90 - \text{valor}) / 1.2816$.

LO QUE NO SIGNIFICA

— El propio código dice que $z \cdot \sigma \cdot \sqrt{L}$ está mal: supone errores normales e independientes por período, y «ninguna de las dos cosas se cumple — la demanda es no negativa y sesgada, y un pronóstico que va alto hoy va alto mañana, así que los errores se acumulan más rápido que $\sqrt{L}$ ». La rama medida existe justamente para no usarla.

— El MOQ es un MÍNIMO, no un múltiplo: una necesidad de 520 con MOQ 500 pide 520, no 1000. Antes pedía 1000.

— NO hay período de revisión. El modelo es de revisión continua, con el disparador reemplazado por la razón de cobertura.

— Los cuatro niveles de servicio históricos (0.90, 0.95, 0.97, 0.99) devuelven z fijado a 3 decimales; cualquier otro nivel se calcula con la aproximación de Acklam. Es una discontinuidad deliberada de $\sim 4e-4$ — los cuatro valores se dejan clavados para que las cifras de un tenant existente no se muevan por debajo de él. No es un error de la aproximación: es el precio de no reescribir el pasado.

— Ojo con leer el archivo: $z[0.90] = 1.282$ y $Q90_Z = 1.2816$ son la MISMA constante redondeada distinto, en el mismo módulo. La primera arma el colchón; la segunda despeja σ del intervalo del pronóstico. No son intercambiables por casualidad, son iguales a propósito.

— El punto de reorden y la señal pueden contradecirse: cuando el stock de seguridad supera el 20% de la demanda del lead time, un SKU puede estar por DEBAJO de su punto de reorden publicado y aun así pintarse OK con cantidad 0.

Lead time aprendido

[backend/inventory/service.py:1217 · get_learned_lead_times](#)

Cada recepción completa de una orden deja una observación del plazo real del proveedor. Con tres observaciones, el promedio aprendido reemplaza a lo configurado.

Aprendizaje

```
AVG(lead_time_days) GROUP BY LOWER(supplier) HAVING COUNT(*) >= 3
```

MIN_LEAD_TIME_OBSERVATIONS = 3. Por debajo, el proveedor ni aparece en el mapa.

LO QUE NO SIGNIFICA

- Lo aprendido GANA incondicionalmente sobre todo lo configurado, y aplica a TODOS los SKUs de ese proveedor, incluso los configurados a mano uno por uno.
- Es una media simple sobre todas las observaciones de la historia: sin ponderación por recencia, sin rechazo de atípicos, sin techo.
- Solo escribe observación una recepción COMPLETA. Una parcial no aporta nada. Y en una orden multi-proveedor, TODOS los proveedores quedan fechados por la finalización de la orden entera.

Multi-bodega y traslados

[backend/inventory/service.py:1852 · get_inventory_status_by_warehouse](#)

La demanda se reparte entre bodegas de dos formas, y cuando una bodega necesita lo que a otra le sobra, se propone un traslado en vez de una compra.

CÓMO FUNCIONA

- Modo «store»: la sesión se entrenó con columna de tienda, así que cada bodega tiene su propio pronóstico real. Participación 1.0.
- Modo «share»: el pronóstico global del SKU se multiplica por la participación configurada de cada bodega. Si ninguna tiene participación, el 100% va a la bodega por defecto.
- Un donante debe quedar con al menos 30 días de cobertura propia y poder cubrir al menos el 80% de la necesidad.
- El carril se acepta si es estrictamente más rápido que comprar Y estrictamente más barato; sin costo unitario conocido se acepta solo por tiempo, y se dice.

LO QUE NO SIGNIFICA

- El estado agregado («Todas») NO es la suma de las señales por bodega: es un cálculo independiente sobre stock sumado contra el pronóstico sin repartir. Las dos vistas pueden discrepar legítimamente en un SKU.
- Un carril sin configurar vale 1 día, costo 0 y fijo 0 — el propio módulo lo llama «deliberadamente optimista». Le gana a casi cualquier compra. La bandera `lane\_is\_default` viaja para que la pantalla lo diga.

El optimizador

[ForecastingCore/forecasting_core/business/optimizer.py:2 · MILP inventory optimizer](#)

Un MILP resuelto con HiGHS (`scipy.optimize.milp`) que reparte compras y traslados en el horizonte, minimizando costo total.

Objetivo

$\min \Sigma \text{holding} \cdot \text{inv} + \text{stockout} \cdot \text{short} + \text{order_cost} \cdot \text{order} + \text{transfer_cost} \cdot \text{transfer} + \text{fixed} \cdot \text{ship}$

Variables por bucket: order, transfer, inv, short (enteras ≥ 0) y ship (binaria, solo en carriles con costo fijo).

Costos

$\text{holding} = \text{unit_cost} \cdot 0.20 / 365 \cdot \text{días_por_bucket}$; $\text{stockout} = \text{order_cost} \cdot 3.0$

0.20 es el porcentaje anual de bodegaje por defecto; 3.0 el multiplicador de quiebre.

LO QUE NO SIGNIFICA

- Límite de 10 segundos de solver, y TODO solve corre en un único hilo dedicado: entrar a HiGHS desde hilos distintos produce un bloqueo que un mutex no arregla.
- Ante excepción, timeout o más de 5000 variables, degrada a un respaldo ingenuo que IGNORA los traslados por completo, y lo reporta como status «fallback».
- Solo un solve concurrente: el segundo recibe 503 y reintenta. Con dos, dos compradores recibían dos planes distintos del mismo catálogo.
- Un costo unitario de 0 o negativo se trata como ausente y se usa 1.0, marcando la línea como `assumed\_unit\_cost`. El `total\_cost` puede ser un número sobre nada.
- El MOQ se aplica DESPUÉS del solve, repartiendo el faltante a la línea más grande. El MILP no tiene variable de pedido mínimo.

Lo que las cifras de negocio no dicen

[backend/inventory/roi_service.py:516](#)

La pantalla de impacto es deliberadamente más pobre de lo que podría ser, y eso es una decisión, no una carencia.

LO QUE NO SIGNIFICA

- `capital\_freed` es la diferencia entre dos fotos mensuales de sobrestock. El sobrestock también baja por vender, por merma, por borrar productos y por reentrenar: la cifra NO es atribuible a Faro y el código lo dice.
- No se calcula ningún titular de «Faro te ahorró \$X» ni un conteo de «quiebres evitados», porque ambos necesitan supuestos que no están fundados en los datos del cliente.
- El stock muerto es una heurística: un SKU está «muerto» cuando su consumo observado es menor al 20% del esperado por pronóstico, con al menos 2 fotos y sin reposición en la ventana.
- El ABC usa demanda diaria $\times$ costo unitario como aproximación de ingreso, cayendo a costo 1.0 cuando no hay costo. Los cortes XYZ son CV 0.5 y 1.0, fijos en el código.

5. La plataforma: multi-tenant, trabajos, configuración y API

Tres capas con una separación estricta, una cola de trabajos que ES una tabla, y aislamiento por tenant que es disciplina de consulta, no una garantía de la base. Este capítulo dice dónde están los candados y también dónde NO están.

Capas y el único límite que se verifica solo

[backend/tests/test_no_pandas_in_backend.py:33](#)

Motor ML, API de orquestación y UI. De las tres reglas, solo una la comprueba una máquina: pandas y numpy no pueden importarse en `backend/` fuera de tres módulos.

Permitidos	todo bajo backend/dataframes/, backend/utils/temporal_agg.py y backend/workers/runner.py
Qué cruza el límite	Entra una ruta o bytes; sale Python plano — list[dict], dict, list[str], con NaN convertido a None
Las cuatro excepciones	read_dataframe, dataframe_from_records, filter_dataframe_by_date y last_row_per_group: los únicos puntos donde un DataFrame llega al motor, cada uno etiquetado como tal

LO QUE NO SIGNIFICA

- NO existe un test equivalente que prohíba lógica de negocio en el frontend ni ML en el backend. Esa mitad es convención.

Multi-tenancy

[backend/auth/guards.py:148](#) · [get_current_user](#)

El tenant viaja en el JWT y se hace cumplir consulta por consulta. Cada función de servicio recibe `tenant\_id` como primer argumento y cada sentencia lleva `AND tenant\_id = %s`.

LO QUE NO SIGNIFICA

- NO hay row-level security en la base. El aislamiento es disciplina, no garantía: un solo `AND tenant\_id` olvidado lo rompe.
- El middleware de contexto de tenant es SOLO observabilidad — su propio docstring lo dice. La autoridad es `CurrentUser.tenant_id`, que nunca se lee del cuerpo de la petición.
- La red de seguridad son tests que leen la realidad: uno recorre la tabla de rutas de FastAPI y falla cualquier mutación sin guardia; otro lee el catálogo de Postgres y exige que toda tabla con `tenant_id` tenga FK con ON DELETE CASCADE (nació de 1,3 millones de filas huérfanas de 24.794 tenants muertos).
- Un id de otro tenant devuelve 404, no 403: no confirma que exista.

Autenticación

[backend/auth/jwt_handler.py](#)

JWT propio, no Supabase. Dos credenciales distintas, una persona y una máquina.

Token de acceso	15 minutos, HS256, claims sub/tenant_id/role/email_verified/jti.
Token de refresco	7 días. NO es un JWT: 64 bytes aleatorios guardados solo como SHA-256. Máximo 5 vivos por usuario.
Revocación	Dos mecanismos: lista negra por jti para el logout, y corte por tiempo (sessions_invalid_before) para el cambio de contraseña, que no puede conocer un jti.
Llaves de API	sk_live_ + 32 bytes. SHA-256 sin sal (32 bytes de entropía no la necesitan). Nunca pueden ser admin: solo viewer o analyst.
Límites de la llave	120 por minuto y el techo diario del plan, decididos bajo un advisory lock — contar y luego insertar dejaba pasar 16 de 20 llamadas contra un techo de 5.

LO QUE NO SIGNIFICA

- REVISAR: `ACCESS\_TOKEN\_EXPIRE\_MINUTES` existe, está documentada y registrada... y nadie la lee. Los 15 minutos son una constante en el código. Cambiar esa variable no hace nada.
- El límite de llaves FALLA ABIERTO ante un error de base: se prefiere servir a bloquear. Es deliberado y está documentado.
- Una llave actúa con SU propio rol, no con el de quien la creó, y toda escritura suya deja fila de auditoría con actor `api\_key:<id>`.

Trabajos y el worker

[backend/training/queue.py:50](#)

La cola es la tabla `jobs`. `enqueue()` es un no-op explícito: insertar la fila ES encolar.

La reclamación

```
UPDATE jobs SET status='RUNNING', worker_id=%s WHERE id = (SELECT id FROM jobs WHERE
status='QUEUED' ORDER BY created_at FOR UPDATE SKIP LOCKED LIMIT 1)
```

SKIP LOCKED es lo que hace que dos réplicas tomen trabajos DISTINTOS. Reemplazó a un leer-luego-escribir que entrenó la misma sesión dos veces.

job-worker	Sondea cada 2 s mientras haya cupo. Máximo de trabajos concurrentes: 2 por proceso.
job-scheduler	Cada 60 s: trabajos programados vencidos, con cron evaluado en la zona horaria del tenant.
integration-sync	06:00 UTC. Antes de las alertas, a propósito.
inventory-alerts	08:00 UTC: digest de quiebres, desvíos de plazo de proveedor y recordatorios de datos viejos.
overstock-snapshot	Día 1, 00:05 UTC: foto de sobrestock y después los correos de recuento mensual. El orden importa.

LO QUE NO SIGNIFICA

- SCHEDULER_ENABLED debe estar activo en EXACTAMENTE UNA instancia o cada alerta diaria sale dos veces.

- La recuperación de huérfanos corre al arrancar el WORKER, no la API: un redesplicue de la API no debe matar entrenamientos vivos de otro proceso.
- REVISAR: el tope de trabajos por proceso es 2 y el tope por plan es 8. En un despliegue de un solo proceso manda el 2.

Estado de la sesión y los seis bloques de configuración

[backend/sessions/state_machine.py:26](#)

DRAFT → ... → MODELS_CONFIGURED → QUEUED → RUNNING → COMPLETED/FAILED. Seis blobs JSONB en `session\_configs` que el runner ensambla en un diccionario para el motor.

columns_cfg	Mapeo de columnas del usuario al esquema canónico.
features_cfg	Rezagos, ventanas, calendario, país de feriados.
models_cfg	Los modelos que el usuario seleccionó — la selección que el enrutamiento solo puede estrechar.
validation_cfg	Partición, cortes, horizonte.
forecast_cfg	Horizonte y cuantiles.
business_cfg	Nivel de servicio 0.95, lead time, 20% de bodegaje, multiplicador de quiebre 3.0.

LO QUE NO SIGNIFICA

- REVISAR: `RUNNING` está en la tabla de transiciones pero NINGÚN código de producción lo pone en una sesión — solo `jobs.status` pasa a RUNNING. El runner usa `force\_status`, que se salta la máquina entera.
- REVISAR: la llave primaria de `session\_configs` es solo `session\_id`; `tenant\_id` es columna, no parte de la llave.
- REVISAR: `business.lead\_time\_days` vale 7 en el runner y 15 en la semilla del asistente. Los dos defaults no coinciden.
- La compuerta de datos (`data\_gate.enforce`) bloquea el LANZAMIENTO, no el guardado, y tiene un solo punto de llamada — lo que cubre entrenar, el demo, la sincronización ERP y el script de semilla a la vez.

Configuración de servicios

[backend/service_config/](#)

Un registro único del que se generan el panel, `.env.example` y la referencia. Cuatro capas de precedencia y el panel dice cuál ganó en cada campo.

CÓMO FUNCIONA

- Precedencia: override del tenant (solo servicios de canal) > override de instancia > entorno > default de Settings.
- Un secreto se cifra con Fernet; sin llave la escritura se RECHAZA con motivo, nunca se degrada a texto plano.
- Una lectura NUNCA lanza: si la base falla, se reporta el entorno y un despliegue solo-.env sigue funcionando.

- Caché de 10 segundos: una llave pegada en el contenedor de API llega al de worker sin reiniciar.

LO QUE NO SIGNIFICA

- Solo por entorno, a propósito: todo el núcleo (una UI que puede reescribir su propia URL de base deja a todos fuera) y la llave Fernet (un panel que pudiera reescribirla volvería ilegibles sus propios secretos).
- `WHATSAPP\_WEBHOOK\_BASE\_URL` es solo de instancia aunque su servicio sea por tenant: el webhook verifica la firma ANTES de saber de qué tenant es el mensaje.
- Arranque en frío: mientras el despliegue tenga EXACTAMENTE UN tenant, sus admins lo operan. Se acaba en dos. Una base inalcanzable no otorga nada.

Superficie pública y forma de los errores

[backend/api/public_surface.py:26](#)

El contrato publicado son ocho pares (método, ruta). Todo lo demás —271 rutas— es alcanzable con una llave pero nadie prometió mantenerlo.

Envoltura de éxito

```
{"success": true, "data": ..., "meta": {"timestamp": ...}}
```

En 33 de 35 módulos.

Envoltura de error

```
{"detail": "<inglés>", "error_code": "<snake_case estable>", "error_params": {...}}
```

El frontend renderiza `errors.<error\_code>` interpolando los params; `detail` es el respaldo.

PUBLIC_API_ONLY

Monta toda la app y después PODA las rutas a la lista pública más /health. Podar después de montar es deliberado: elegir routers dejaría colarse a un vecino interno.

400 malformed_path

Un NUL en la ruta se rechaza de entrada; antes llegaba a psycpg2 y salía como 500.

503 server_busy

Pool de conexiones agotado, con Retry-After. Ocupado no es roto.

422 validation_error

Con el input saneado: un NaN en el eco hacía que Starlette convirtiera el 422 en un 500 sin cuerpo.

LO QUE NO SIGNIFICA

- PUBLIC_API_ONLY no cambia nada de autenticación ni permisos: es menos alcance, no un segundo modelo de seguridad. Y NO aísla la base — las dos instancias comparten un Postgres.
- REVISAR: los conteos de rutas en los comentarios (246, 261) están desactualizados; hoy hay 271 decoradores.

El frontend

[Frontend/src/](#)

Next.js 14 App Router, 24 páginas con rutas en español, un cliente de API de 226 funciones y tres catálogos de i18n con formas distintas a propósito.

translations.ts	3.034 claves por idioma, búsqueda en tiempo de ejecución. Un typo devuelve la clave.
landing.ts / serviceConfig.ts	Tipados: una traducción faltante es error de COMPILACIÓN, que es la garantía que translations.ts necesita un script para dar.
Proxy	/api/:path* → \${BACKEND_URL}/api/v1/:path*. Ojo: el navegador llama /api/... SIN el v1; el proxy lo agrega.
BACKEND_URL	Por defecto 127.0.0.1:8010 — IPv4 explícito, porque Node resuelve localhost a ::1 y uvicorn escucha en IPv4, produciendo un 500 sin cuerpo.

LO QUE NO SIGNIFICA

- NO HAY TESTS DE FRONTEND. Cero. `tsc --noEmit` verifica tipos, no comportamiento. Lo que hay son tres guiones de Playwright que se corren a mano.
- Las rutas en español son canónicas; las inglesas viejas redirigen con 308 permanente.
- Los rewrites se evalúan en tiempo de CONSTRUCCIÓN: una imagen Docker debe construirse con BACKEND_URL puesta.